PROJECTE ERP · LLIURAMENT DE DISSENY PREVI

LudusERP

Sistema de gestió empresarial per a una botiga de videojocs digitals

Document de disseny UML, model de dades i requeriments funcionals, previ a la implementació amb Spring Boot, Thymeleaf, MySQL i MongoDB.

Projecte intermodular · Desenvolupament d'Aplicacions Multiplataforma

Versió del document: 1.2 · **Data:** 26 d'octubre de 2026

Grup: _____

Índex

1. Descripció general del projecte
    - 1.1 Què és LudusERP
    - 1.2 El negoci té dues cares
    - 1.3 Abast: mòduls del sistema
    - 1.4 Decisions de disseny preses i per què
    - 1.5 No-objectius
2. Perfils d'usuari i model de permisos
    - 2.1 Els rols del sistema
    - 2.2 Matriu de permisos
3. Requeriments funcionals i com els resol el disseny
    - 3.1 Perfils d'usuari
    - 3.2 Mòduls CRUD
    - 3.3 Generació d'informes en PDF
    - 3.4 Multiidioma (i18n)
    - 3.5 Comunicació amb WebSockets
    - 3.6 Treballs en segon pla i threads
    - 3.7 Seguretat i xifrat
4. Requeriments tècnics (no funcionals)
5. Diagrama de casos d'ús
    - 5.1 Accés sense sessió
    - 5.2 Client
    - 5.3 Gestor de productes
    - 5.4 Gestor de clients
    - 5.5 Gestor d'inventari
    - 5.6 Administrador
    - 5.7 Sistema (processos automàtics)
6. Diagrama de classes
    - 6.1 Visió per mòduls
    - 6.2 Seguretat i usuaris
    - 6.3 Catàleg
    - 6.4 Aprovisionament: l'ordre de compra
    - 6.5 Inventari: les claus de llicència
    - 6.6 Vendes i facturació
7. Diagrama d'arquitectura
8. Model de la base de dades MySQL
    - 8.1 Seguretat i usuaris
    - 8.2 Catàleg: fitxa i imatges
    - 8.3 Catàleg: classificació
    - 8.4 Aprovisionament: ordres de compra
    - 8.5 Inventari: claus de llicència
    - 8.6 Vendes i facturació
9. Model de dades a MongoDB
10. Traçabilitat requeriment → disseny
11. Repartiment de tasques

1. Descripció general del projecte

1.1 Què és LudusERP

LudusERP és el sistema de gestió interna d'una botiga de videojocs digitals: una plataforma de venda en línia d'estil Steam o Epic Games Store, però vista des de dins de l'empresa. La part que el client final veu —el catàleg, la fitxa del joc, la cistella i el pagament— és només la façana; el que es desenvolupa aquí és tota la maquinària que hi ha al darrere, de la mateixa manera que Odoo no és una botiga sinó l'eina amb què es governa una botiga.

La diferència clau respecte d'un ERP de comerç tradicional és que **el producte no és físic**. No hi ha magatzem, ni caixes, ni transport: el que s'inventaria són **claus de llicència**, codis alfanumèrics que l'empresa compra a l'engròs al distribuïdor o a l'estudi que publica el joc i revèn d'un en un al client. Això fa que el concepte d'estoc segueixi existint —es poden exhaurir les claus d'un joc concret— però que sigui informació pura, i per tant un problema de bases de dades i de control de concurrència en comptes d'un problema de logística.

1.2 El negoci té dues cares

Tot el disseny s'organitza al voltant d'un cicle que entra per un extrem i surt per l'altre. Entendre aquestes dues cares és el que explica per què existeix cada classe del diagrama:

- **Cara d'aprovisionament (compra).** L'empresa negocia amb un *proveïdor* i li emet una *ordre de compra*: tantes unitats d'aquest joc a tant per unitat. Quan el proveïdor serveix la comanda, aquelles unitats es materialitzen com a *claus de llicència* individuals que entren a l'inventari amb estat «disponible». Aquesta cara és la que genera **cost** i la que alimenta l'estoc.
- **Cara de venda.** Un client afegeix jocs a la cistella i confirma la compra. En pagar, el sistema *reserva i assigna* una clau disponible per cada línia de la comanda, la marca com a venuda, l'afegeix a la biblioteca del client i emet la factura en PDF. Aquesta cara és la que genera **ingrés** i la que consumeix estoc.

La diferència entre el preu de venda d'una clau i el que va costar comprar-la és el **marge**, i és precisament el que l'administrador vol veure al mòdul d'estadístiques. Com que cada clau sap de quina línia de compra prové i a quina línia de comanda ha anat, el marge es pot calcular de veritat i no estimar: és la traçabilitat completa d'una unitat des del proveïdor fins al client.

> **LA IDEA EN UNA FRASE**
>
> Un ERP que compra llicències de videojocs a l'engròs, les custodia com a inventari traçable, les ven al detall a través d'una botiga en línia, i dona a cada perfil de treballador exactament la finestra del procés que li toca gestionar.

1.3 Abast: mòduls del sistema

Mòdul	Responsabilitat	Qui l'opera
Seguretat i usuaris	Registre, autenticació, rols, permisos, empleats i departaments.	Administrador
Catàleg	Videojocs, categories, plataformes, desenvolupadors, imatges de portada i galeria, i preus.	Gestor de productes
Aprovisionament i inventari	Proveïdors, ordres de compra, recepció de mercaderia, generació i custòdia de claus de llicència, control d'estoc.	Gestor d'inventari
Vendes i facturació	Cistella, comandes, pagament, assignació de claus, biblioteca del client, factures en PDF.	Client / Sistema
Analítica	Estadístiques de vendes i de compres, marge, rànquings i alertes d'estoc baix.	Administrador

1.4 Decisions de disseny preses i per què

Aquestes són les decisions que condicionen la resta del document. Es fan explícites aquí perquè es puguin discutir i, si cal, revertir abans d'escriure codi.

> **DECISIÓ 1 — L'ESTOC SÓN CLAUS INDIVIDUALS, NO UN COMPTADOR**
>
> Es podria haver modelat l'inventari com un simple camp `unitats_disponibles` dins de `Videojoc`. S'ha descartat: en aquest negoci la unitat venuda *és* un codi concret que s'ha de lliurar al client, i si no existeix com a entitat no es pot saber quina clau té cada client, ni revocar una clau defectuosa, ni calcular el cost real d'una venda. Modelant `ClauLlicencia` com a entitat, l'estoc d'un joc passa a ser el nombre de claus en estat `DISPONIBLE`, i això dona traçabilitat unitat per unitat.

> **DECISIÓ 2 — UN SOL USUARI, AMB PERFILS 1:1 PENJATS**
>
> Hi ha una única taula d'usuaris i un únic formulari de login per a tothom; el que diferencia un client d'un treballador és el **rol**. Però l'enunciat demana un CRUD d'*Empleats–Departaments*, i un client no té departament. La solució és no heretar: de `Usuari` en pengen `PerfilClient` i `PerfilEmpleat` en relació 1:1 opcional. Així es manté una sola taula d'autenticació i, alhora, existeix la relació Empleat–Departament sense deixar camps buits a la meitat de les files.

> **DECISIÓ 3 — ELS PERMISOS SÓN GRANULARS; EL ROL NOMÉS ÉS UN PAQUET DE PERMISOS**
>
> Si el control d'accés es fes amb `hasRole('GESTOR_PRODUCTES')`, crear un rol nou obligaria a tocar i recompilar el codi. Amb el model `Usuari` – `Rol` – `Permis`, Spring Security carrega com a *authorities* els codis de permís (`PRODUCTE_EDITAR`, `COMPRA_CREAR`…) i les anotacions queden `@PreAuthorize("hasAuthority('PRODUCTE_EDITAR')")`. Afegir un permís nou o reagrupar-los en un rol diferent és inserir files a `rol_permis`: no cal tocar codi ni tornar a desplegar.

> **DECISIÓ 4 — RES NO ÉS PÚBLIC: CAL SESSIÓ FINS PER VEURE EL CATÀLEG**
>
> S'havia plantejat un catàleg obert, com a qualsevol botiga real, amb el login només a l'inici del pagament. S'ha descartat: com que l'avaluació del projecte se centra en el control d'accés i no en la captació de clients, surt més net que **tota** la informació quedi darrere de l'autenticació. La `SecurityFilterChain` només deixa passar sense sessió la pantalla d'accés, el registre i els recursos estàtics; qualsevol altra URL redirigeix al login. No hi ha, doncs, rol de visitant.
>
> La conseqüència tècnica és que **desapareix la cistella anònima** i amb ella la fusió de cistelles en autenticar-se: una `Cistella` sempre penja d'un `Usuari` i el camp `usuari_id` no admet nuls. És una peça menys a implementar i un cas de prova menys a cobrir.

1.5 No-objectius

Marcar el que el projecte *no* fa evita que l'abast creixi sense control i deixa clar què no s'ha d'avaluar:

- **No hi ha passarel·la de pagament real.** El pagament se simula amb un servei intercanviable que retorna autoritzat o rebutjat; integrar Stripe o Redsys queda fora.
- **No hi ha descàrrega ni execució de jocs.** El que es lliura al client és la clau de llicència, no un binari. No es gestionen fitxers de joc ni DRM.
- **No hi ha comptabilitat oficial.** S'emeten factures i s'expliquen marges, però no es porten llibres comptables ni es presenten impostos.
- **No és multiempresa.** La instància gestiona una sola botiga.

2. Perfils d'usuari i model de permisos

2.1 Els rols del sistema

L'enunciat demana un mínim de tres perfils. Aquí n'hi ha cinc, perquè el nivell intermedi («usuari avançat» o gestor) s'ha dividit en tres especialitzacions que es corresponen amb departaments reals d'una botiga: qui porta el catàleg no és qui porta les compres ni qui atén els clients. Tots són files de la taula `rol`; no hi ha cap jerarquia de classes Java al darrere.

Rol	Nivell	Què pot fer
`CLIENT`	Bàsic	És el perfil mínim amb què es pot veure producte: consulta i cerca al catàleg, obre fitxes i galeries, cistella persistent, compra, biblioteca de claus, descàrrega de factures i tria d'idioma.
`GESTOR_PRODUCTES`	Avançat	CRUD de videojocs, categories, plataformes i desenvolupadors. Puja i ordena imatges, designa la portada i modifica preus. No veu dades de clients ni de compres.
`GESTOR_CLIENTS`	Avançat	Consulta el llistat i la fitxa dels clients amb el seu historial de comandes i les claus que tenen a la biblioteca. Només lectura: no pot crear ni esborrar clients.
`GESTOR_INVENTARI`	Avançat	CRUD de proveïdors, creació i confirmació d'ordres de compra, recepció de mercaderia amb generació de claus, anul·lació de claus defectuoses i vigilància de l'estoc.
`ADMIN`	Administrador	Tot l'anterior, més: CRUD d'empleats i departaments, assignació de rols als empleats, estadístiques de vendes i compres i configuració del sistema.

2.2 Matriu de permisos

Aquesta matriu és el contingut inicial de la taula pont `rol_permis`. És la font de veritat del control d'accés: cada cel·la marcada és una fila a la base de dades i es tradueix directament en una *authority* de Spring Security.

Permís	CLIENT	G. PRODUCTES	G. CLIENTS	G. INVENTARI	ADMIN
`CATALEG_LLEGIR`	✓	✓	✓	✓	✓
`PRODUCTE_CREAR / EDITAR / ESBORRAR`	–	✓	–	–	✓
`PRODUCTE_IMATGE_GESTIONAR`	–	✓	–	–	✓
`PREU_MODIFICAR`	–	✓	–	–	✓
`COMANDA_CREAR` (comprar)	✓	–	–	–	–
`COMANDA_PROPIA_LLEGIR`	✓	–	–	–	–
`CLIENT_LLISTAR / LLEGIR`	–	–	✓	–	✓
`PROVEIDOR_CRUD`	–	–	–	✓	✓
`COMPRA_CREAR / CONFIRMAR / REBRE`	–	–	–	✓	✓
`STOCK_CONSULTAR`	–	✓	–	✓	✓
`EMPLEAT_CRUD` / `DEPARTAMENT_CRUD`	–	–	–	–	✓
`ROL_ASSIGNAR`	–	–	–	–	✓
`ESTADISTICA_VEURE`	–	–	–	–	✓
`CONFIG_EDITAR`	–	–	–	–	✓

3. Requeriments funcionals i com els resol el disseny

Cada apartat recull el requeriment tal com el demana l'enunciat i, tot seguit, la decisió concreta amb què el projecte el satisfà. L'objectiu és que no quedi cap requeriment sense una peça del disseny assignada.

3.1 Perfils d'usuari

Requeriment. Mínim tres perfils amb permisos i funcionalitats diferenciades: administrador, usuari avançat o gestor, i usuari bàsic.

Com es resol. Se'n defineixen cinc, detallats a l'apartat 2: l'usuari bàsic és el **client**, el nivell avançat es desplega en **gestor de productes**, **gestor de clients** i **gestor d'inventari**, i per damunt hi ha l'**administrador**. No hi ha perfil de visitant: sense sessió només es pot entrar o registrar-se (decisió 4). La diferenciació no s'implementa amb herència de classes sinó amb el triangle `Usuari` – `Rol` – `Permis`, de manera que els permisos són dades i no codi.

3.2 Mòduls CRUD

Requeriment. Mínim quatre mòduls CRUD amb taules relacionades a MySQL, cadascun amb llistat paginat o filtrat, vista de detall, alta, edició, eliminació i validacions tant de servidor com de client.

Com es resol. Se'n lliuren **sis**, tots sobre entitats relacionades entre elles i no illes independents:

CRUD	Taules relacionades	Filtres i particularitats del llistat
Videojocs	`videojoc` – `categoria` (N:M) – `plataforma` (N:M) – `desenvolupador` (1:N) – `imatge_videojoc` (1:N)	Cerca per títol, filtre per categoria, plataforma, rang de preu i actiu/inactiu. Paginació de 20. Columna d'estoc calculat.
Proveïdors	`proveidor` – `compra` (1:N)	Cerca per nom comercial o CIF, filtre per país i per actiu. Mostra el total comprat acumulat.
Ordres de compra	`compra` – `linia_compra` (1:N) – `videojoc` – `clau_llicencia` (1:N)	Filtre per proveïdor, estat i rang de dates. Capçalera amb línies editables; en rebre-la es generen les claus.
Clients	`usuari` – `perfil_client` (1:1) – `comanda` (1:N)	Cerca per email o nom, filtre per país i per data d'alta. Detall amb historial de comandes. Lectura per al gestor de clients.
Empleats i departaments	`usuari` – `perfil_empleat` (1:1) – `departament` (1:N) – `rol` (N:M)	Filtre per departament i per rol. L'alta d'empleat crea l'usuari, li assigna rols i envia la contrasenya inicial per correu.
Comandes	`comanda` – `linia_comanda` (1:N) – `clau_llicencia` – `factura` (1:1)	Filtre per estat, client i dates. El client només hi veu les seves (filtre forçat a la capa de servei).

Validacions. Al servidor s'usa Bean Validation sobre els DTO (`@NotBlank`, `@Email`, `@Positive`, `@Size`) més validadors propis per a les regles de negoci que una anotació no pot expressar: que no es pugui esborrar un videojoc amb claus venudes, que el preu de venda no quedi per sota del cost mitjà, o que no es pugui esborrar un departament amb empleats a dins. Al client, atributs HTML5 (`required`, `min`, `pattern`) i verificació en JavaScript abans d'enviar, entenent que és comoditat per a l'usuari i no seguretat: la validació de servidor sempre es torna a executar.

3.3 Generació d'informes en PDF

Requeriment. Com a mínim un PDF generat automàticament, amb format presentable i professional.

Com es resol. Se'n generen tres, tots amb la mateixa canonada tècnica: una plantilla Thymeleaf d'HTML i CSS que es renderitza i es converteix a PDF amb **OpenHTMLtoPDF**. Reaprofitar Thymeleaf, que ja és el motor de plantilles del projecte, evita aprendre una API de dibuix i permet dissenyar el document amb CSS.

- **Factura de venda.** S'emet sola quan una comanda passa a pagada. Porta número correlatiu, dades fiscals, línies amb preu unitari, base imposable, IVA i total. Es desa al disc i queda enllaçada des de la comanda perquè el client la pugui tornar a descarregar.
- **Albarà de recepció.** El genera el gestor d'inventari en rebre una ordre de compra: què s'ha demanat, què ha arribat i quantes claus s'han donat d'alta.
- **Informe mensual de vendes.** El produeix una tasca programada el dia 1 de cada mes: ingressos, cost, marge, rànquing dels deu jocs més venuts i gràfic de l'evolució diària.

3.4 Multiidioma (i18n)

Requeriment. Aplicació disponible en català, castellà i anglès, amb la configuració d'idioma emmagatzemada a MongoDB i els textos servits amb plantilles internacionals.

Com es resol. Els textos no viuen en fitxers `.properties` sinó a la col·lecció `traduccions` de MongoDB, amb documents del tipus `{ clau: "cataleg.afegir_cistella", idioma: "ca", text: "Afegeix a la cistella" }`. Al damunt s'implementa un `MessageSource` propi que llegeix d'aquesta col·lecció amb memòria cau, de manera que les etiquetes `th:text="#{clau}"` de Thymeleaf funcionen igual que sempre però el text es pot corregir sense recompilar ni reiniciar.

La tria de l'idioma es resol en tres nivells, de més específic a més general: la **preferència de l'usuari** desada a `preferencies_usuari`; si no n'hi ha (primer accés, o la mateixa pantalla de login), la **capçalera** `Accept-Language` del navegador; i si tampoc serveix, l'**idioma per defecte del sistema** definit a `config_sistema`. El selector de la capçalera escriu la preferència a MongoDB per als usuaris autenticats; a la pantalla d'accés, l'única sense sessió, es desa en una galeta. Si falta una clau en un idioma, es recorre al català i es deixa un avís al registre perquè la traducció que manca es pugui detectar.

3.5 Comunicació amb WebSockets

Requeriment. Com a mínim una funcionalitat amb sockets TCP o WebSockets.

Com es resol. S'usen **WebSockets amb STOMP** sobre Spring (`spring-boot-starter-websocket`) i SockJS com a pla alternatiu per a navegadors que els bloquegin. Es cobreixen dues funcionalitats i no una, perquè totes dues surten del mateix muntatge:

- **Notificacions en temps real** a la safata de l'empleat (`/user/queue/notificacions`): estoc esgotat, estoc per sota del llindar i ordre de compra rebuda.
- **Estoc viu a la fitxa del producte** (`/topic/stock/{idVideojoc}`): quan una venda deixa un joc sense claus, la pàgina de qui l'estava mirant es passa a «exhaurit» sense refrescar. És l'exemple més clar d'informació que s'actualitza sense recarregar.

3.6 Treballs en segon pla i threads

Requeriment. Incloure threads o processos asíncrons: tasques programades, enviament de correus, processament pesat o importació massiva.

Treball	Mecanisme	Descripció
Correu de confirmació amb factura	`@Async` + `ThreadPoolTaskExecutor`	En pagar, el client no ha d'esperar que el servidor de correu respongui: es llança en un fil del pool i la resposta HTTP torna de seguida.
Recàlcul d'estadístiques	`@Scheduled` (cada nit)	Agrega vendes, costos i marges del dia i desa el resultat a `estadistiques_cache` a MongoDB, perquè el panell de l'administrador no recalculi sobre tota la història a cada visita.
Informe mensual en PDF	`@Scheduled` (cron, dia 1)	Genera l'informe del mes tancat i l'envia per correu a l'administrador.
Vigilància d'estoc baix	`@Scheduled` (cada hora)	Cerca jocs per sota del llindar de claus disponibles i avisa el gestor d'inventari per WebSocket i per correu.
Purga de cistelles abandonades	`@Scheduled` (diari)	Allibera les reserves de claus de cistelles inactives més de set dies perquè l'estoc torni a estar disponible.

3.7 Seguretat i xifrat

Requeriment. Autenticació i control d'accés amb Spring Security, hash de contrasenyes, xifrat d'alguna dada sensible i gestió correcta de sessions i rols.

Mesura	Implementació
Autenticació	Spring Security amb formulari de login, `UserDetailsService` propi que carrega l'usuari i hi projecta els codis de permís dels seus rols com a *authorities*.
Autorització	Doble barrera: regles per URL a la `SecurityFilterChain` i `@PreAuthorize` a mètode de servei, amb `hasAuthority` sobre el permís fi. Un salt directe a una URL d'administració no serveix de res perquè la capa de servei també el rebutja.
Contrasenyes	`BCryptPasswordEncoder` amb factor de cost 12. Mai es desen ni es registren en clar, i el camp no es retorna en cap DTO.
Dades sensibles xifrades	**AES-256-GCM** aplicat de manera transparent amb un `AttributeConverter` de JPA, de manera que el xifrat i desxifrat passen en entrar i sortir de la base de dades i el codi de negoci no se n'adona. S'hi protegeixen: el **NIF** de l'empleat, l'**IBAN** de l'empleat i del proveïdor, el **CIF** del proveïdor i el **codi de la clau de llicència**, que és literalment diners i no ha de quedar llegible per a qui accedeixi al bolcat de la base de dades. La clau mestra es llegeix d'una variable d'entorn i no es versiona.
Sessions	Cookie `HttpOnly` i `SameSite=Lax`, regeneració de l'identificador de sessió en autenticar-se per evitar la fixació de sessió, caducitat per inactivitat i límit d'una sessió simultània per compte d'empleat.
Altres	Protecció CSRF activa als formularis, escapat automàtic de Thymeleaf contra XSS, consultes parametritzades via JPA contra injecció SQL, i limitació d'intents de login per evitar la força bruta.

4. Requeriments tècnics (no funcionals)

Àmbit	Decisió
Arquitectura	Spring Boot 3 amb Java 21, Thymeleaf com a motor de plantilles, MySQL 8 per a les dades relacionals i MongoDB per a configuració, traduccions, registres i memòria cau d'estadístiques. Estructura per **paquets de mòdul de negoci** (`seguretat`, `cataleg`, `aprovisionament`, `vendes`, `analitica`) i dins de cada mòdul les capes.
Patrons	**MVC** obligatori (controlador, servei, repositori, vista). A més: *Repository* via Spring Data, *DTO* amb MapStruct per no exposar entitats a la vista, *Strategy* per a la passarel·la de pagament intercanviable, *Observer* amb els esdeveniments de Spring per desacoblar els efectes d'una venda (correu, factura, estadística), i *Builder* a la construcció dels informes.
Qualitat del codi	Noms de classes i mètodes en anglès i consistents, JavaDoc a totes les classes de servei públiques i a les regles de negoci no evidents, i comentaris només allà on el per què no es dedueix del codi.
Rendiment	Paginació obligatòria a tots els llistats, índexs a les columnes de cerca i de clau forana, càrrega mandrosa amb consultes explícites per evitar el problema de l'N+1, memòria cau de traduccions i d'estadístiques, i reserva de claus amb bloqueig pessimista per no vendre dues vegades la mateixa clau.
Errors i registres	`@ControllerAdvice` global amb pàgines d'error pròpies, excepcions de negoci pròpies, SLF4J amb Logback a fitxer rotatiu per als errors tècnics, i registre d'auditoria a MongoDB (`logs_activitat`) per a tota acció que modifiqui dades, amb qui, què, quan i des de quina adreça.
Interfície	Bootstrap 5 amb una plantilla base de Thymeleaf i fragments reutilitzats (capçalera, menú segons rol, taula paginada, formulari). Disseny sobri però coherent entre pantalles i utilitzable en pantalla petita.
Documentació final	Guia d'instal·lació amb Docker Compose per a MySQL i MongoDB, manual d'usuari per perfil, aquest document d'arquitectura i decisions, i el repartiment de tasques de l'apartat 11.

5. Diagrama de casos d'ús

Es presenta un diagrama per actor en comptes d'un únic diagrama global. La raó és pràctica: amb set actors i una trentena de casos d'ús, un sol diagrama obligaria a reduir-lo tant que no es podria llegir en paper. La unió d'aquestes set vistes és el diagrama de casos d'ús complet. Els cercles foscos són actors, els rectangles arrodonits són casos d'ús i els de fons groc són casos d'ús inclosos per un altre.

5.1 Accés sense sessió

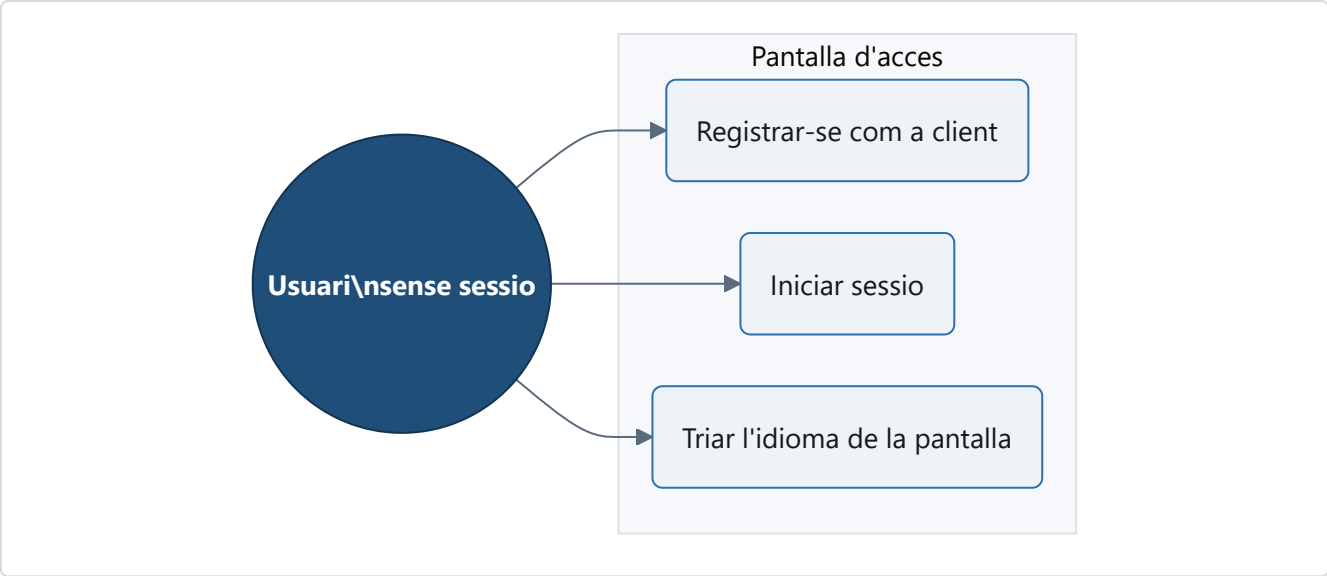


Figura 1. L'únic que es pot fer sense sessió. El catàleg no hi apareix: qualsevol altra URL redirigeix a l'accés (decisió 4).

5.2 Client

Com que res no és públic, el client és el primer actor que veu producte: el catàleg i les fitxes ja són casos d'ús seus. El cas d'ús *Realitzar la compra* inclou tres passos que sempre s'executen i que el client no dispara per separat.

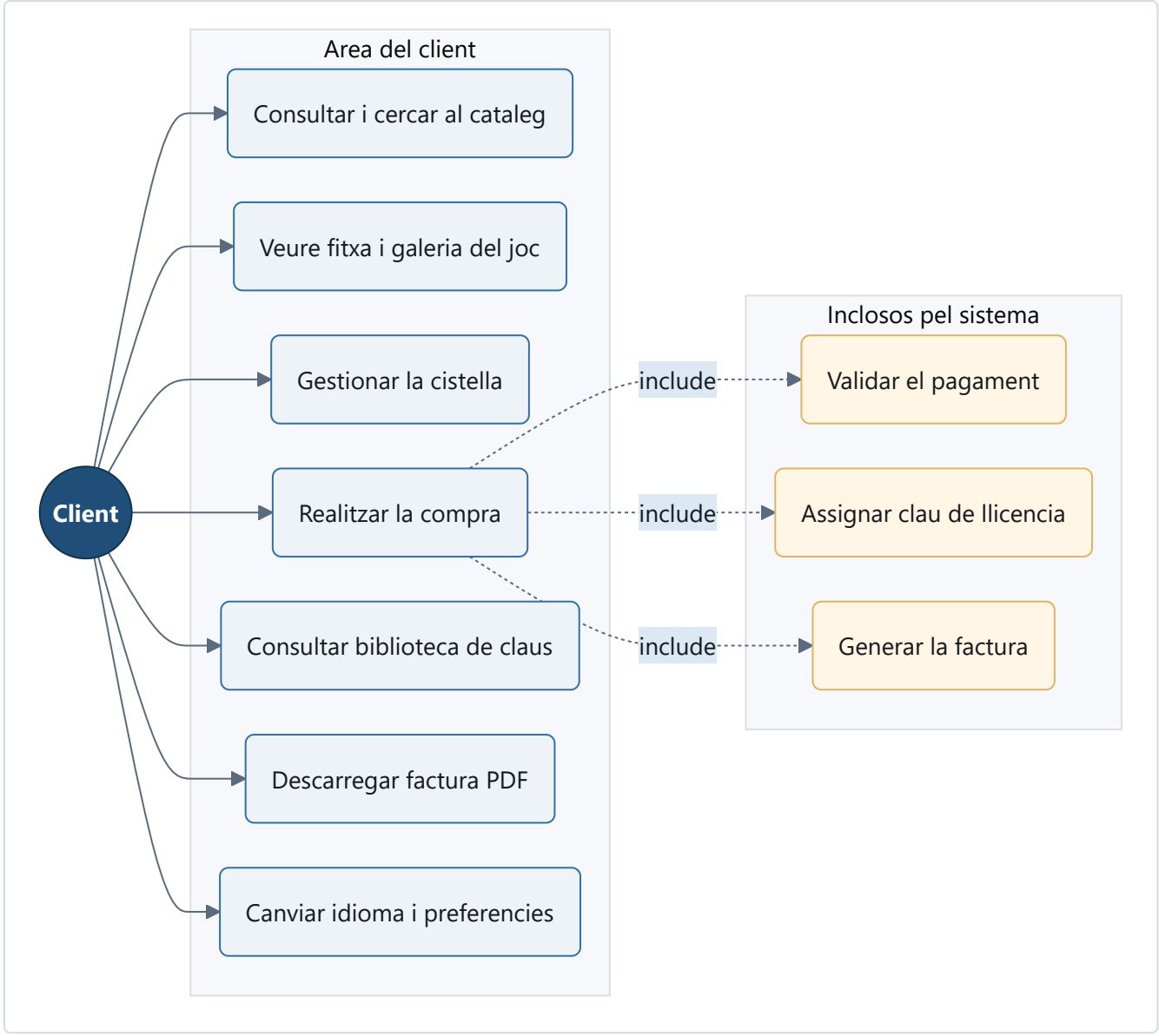


Figura 2. Casos d'ús del client. El catàleg hi apareix perquè ja no és públic. Realitzar la compra inclou els tres passos grocs, que el client no dispara per separat.

5.3 Gestor de productes

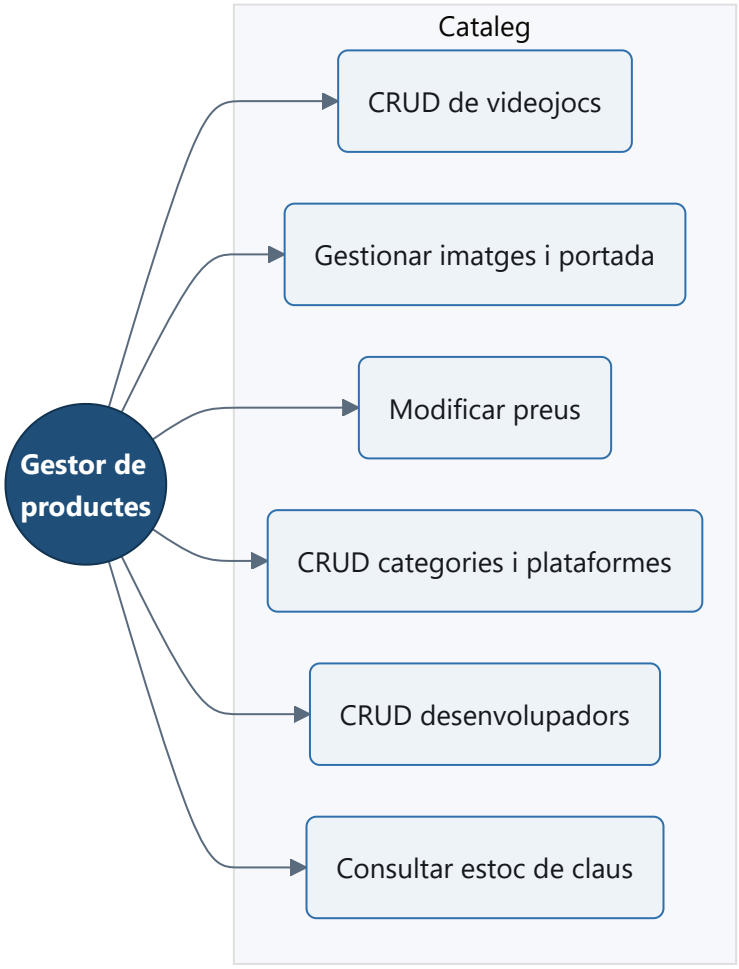


Figura 3. Casos d'ús del gestor de productes. No té accés a dades de clients ni a l'àrea de compres.

5.4 Gestor de clients

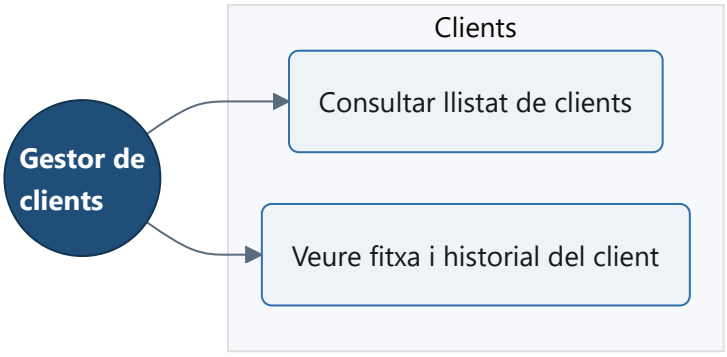


Figura 4. Casos d'ús del gestor de clients. Sobre el client només té lectura: no en pot crear ni esborrar.

5.5 Gestor d'inventari

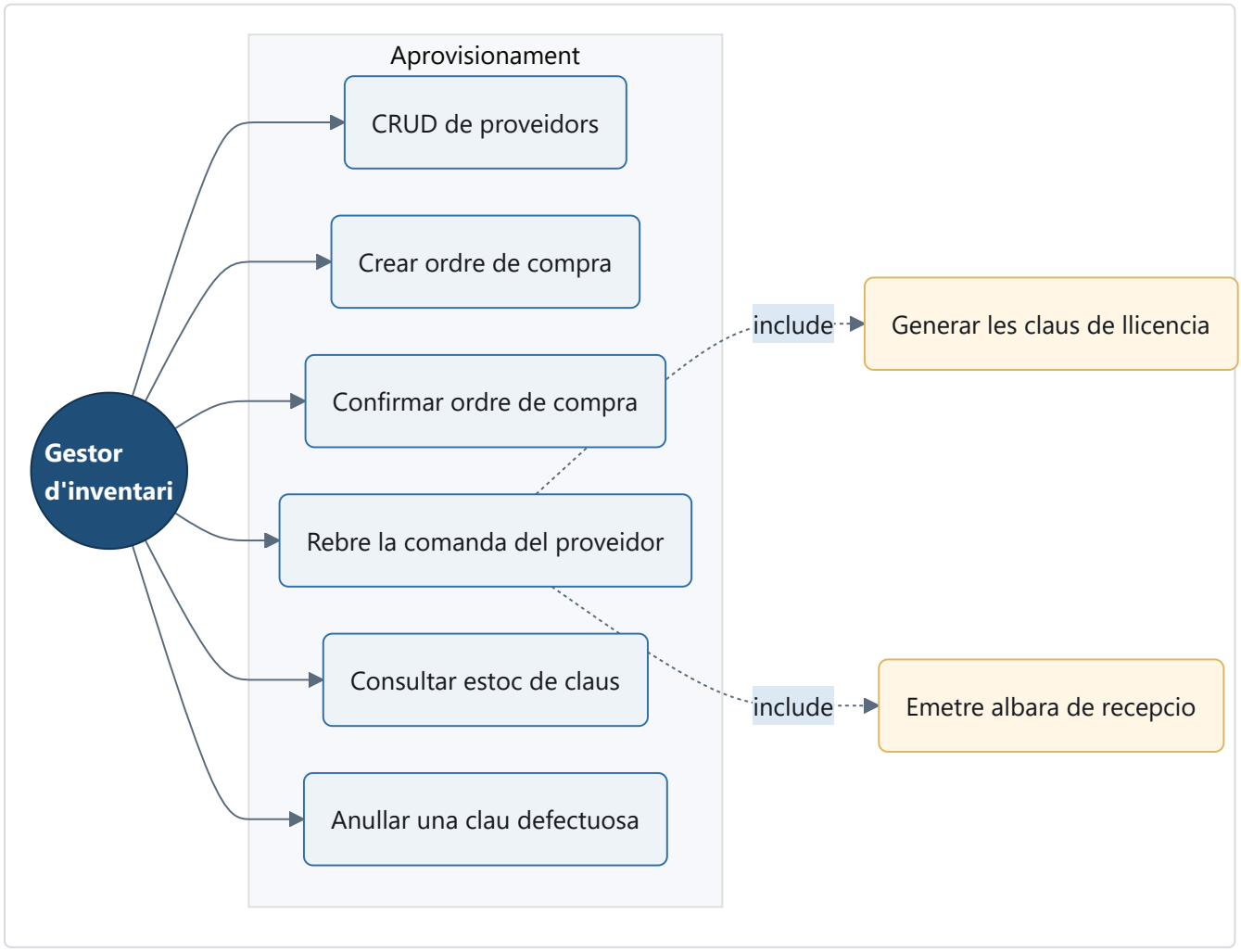


Figura 5. Casos d'ús del gestor d'inventari. Rebre la comanda és l'acte que converteix unitats comprades en claus venibles.

5.6 Administrador

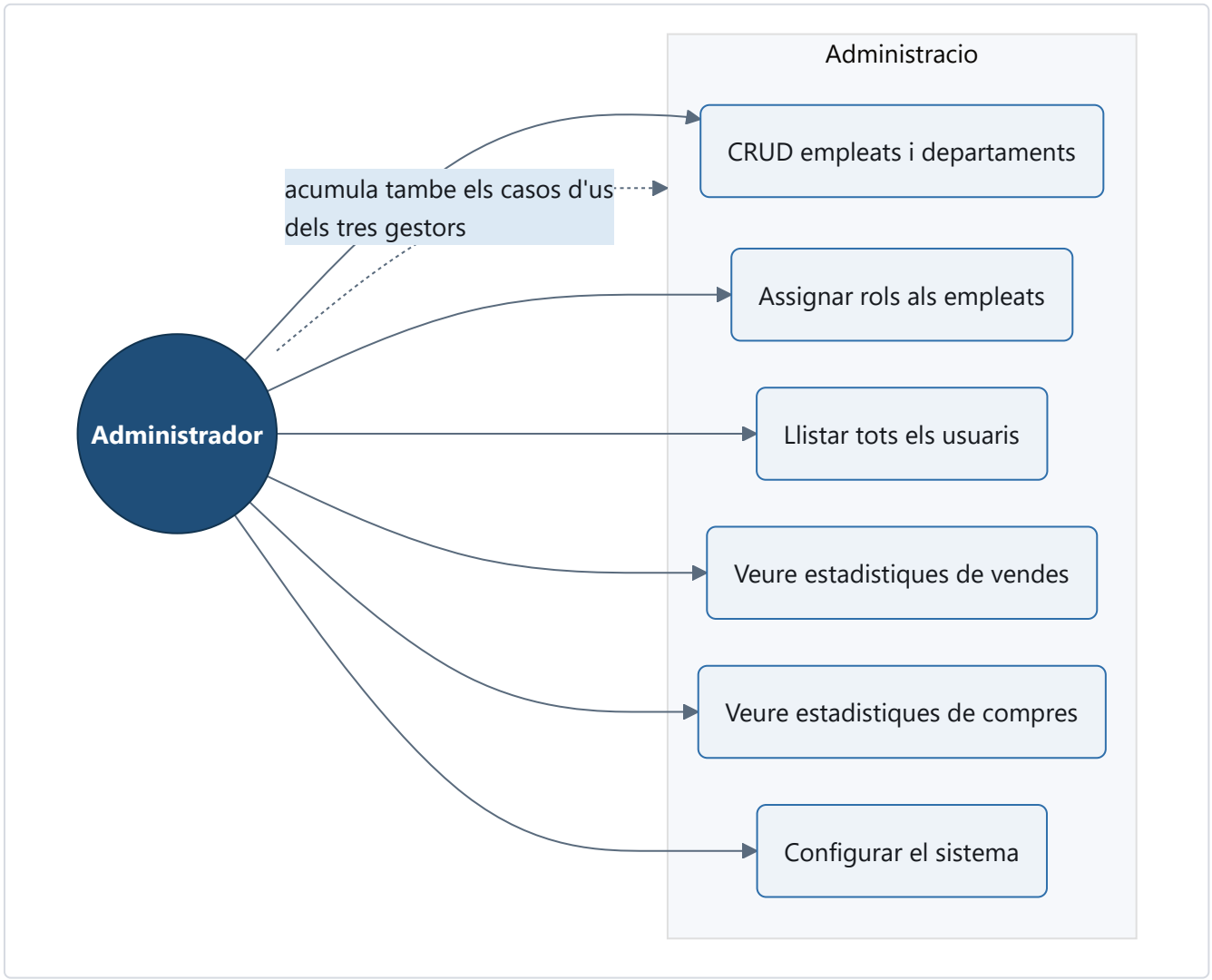


Figura 6. Casos d'ús exclusius de l'administrador. A més d'aquests, té tots els dels tres perfils de gestor.

5.7 Sistema (processos automàtics)

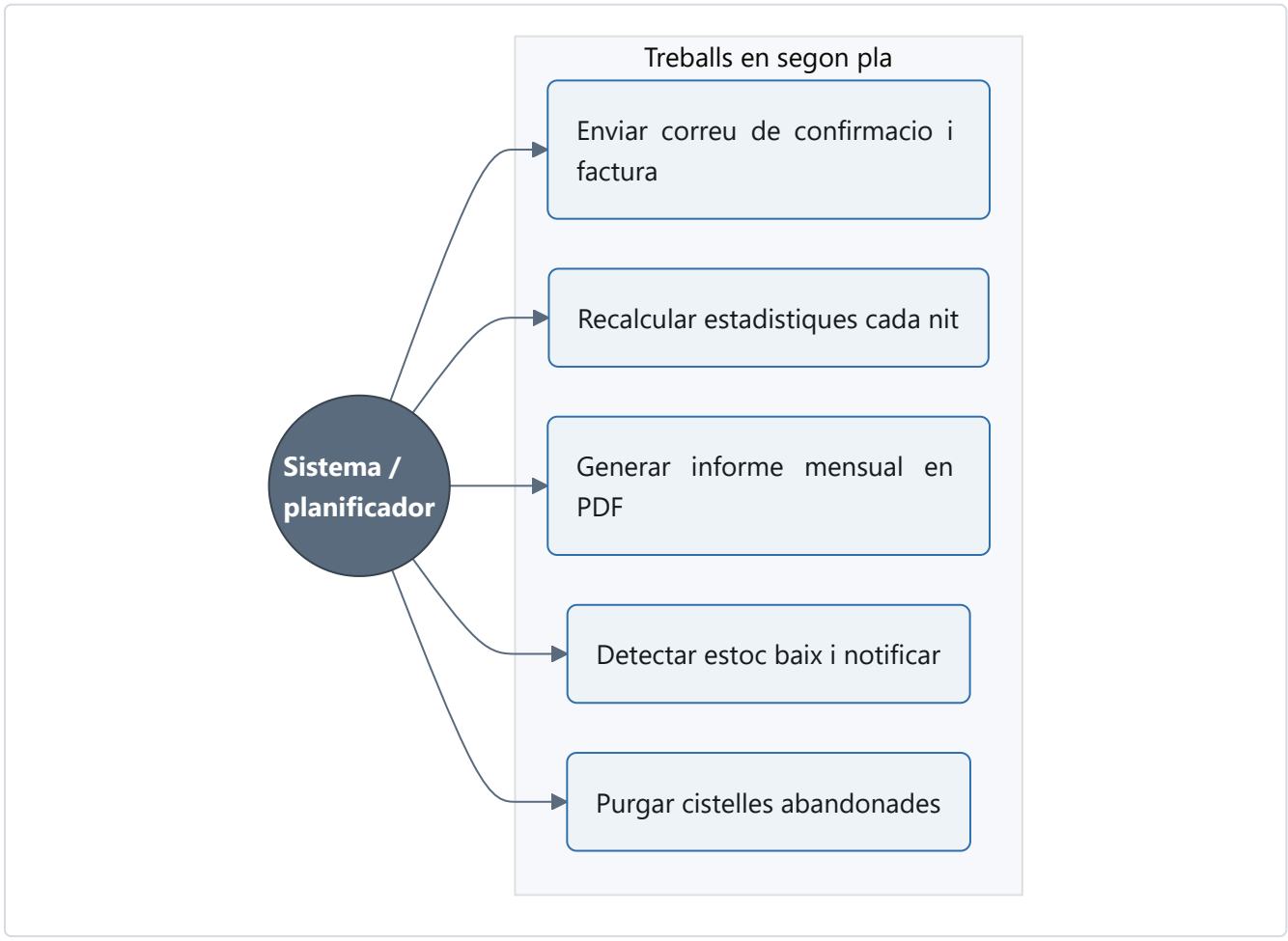


Figura 7. Casos d'ús que arrenca el sistema sense intervenció humana. Corresponen al requeriment 3.6 de treballs en segon pla.

6. Diagrama de classes

El model de domini es presenta primer com un mapa de mòduls i després mòdul per mòdul, amb els atributs i les operacions rellevants. Es divideix així perquè un únic diagrama amb les vint classes del sistema seria il·legible en paper; la unió de les sis vistes és el diagrama de classes complet. Les classes dibuixades amb només una o dues línies (`Usuari`, `Videojoc`, `LiniaCompra`) hi apareixen com a referència perquè són el punt de contacte amb un altre mòdul, on es defineixen senceres.

6.1 Visió per mòduls

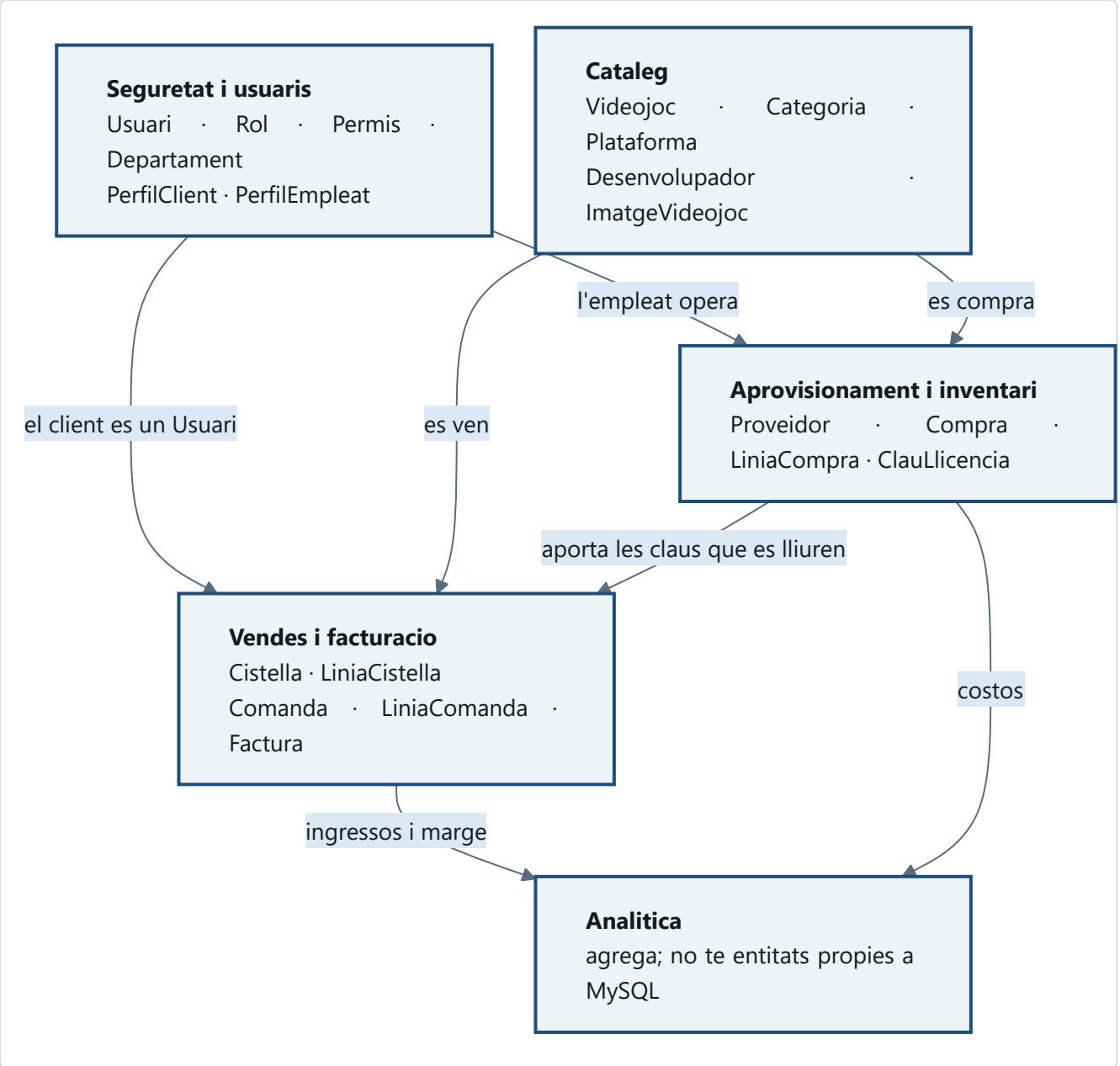


Figura 8. Mòduls del domini i dependències entre ells. La fletxa clau és la que va d'aprovisionament a vendes: sense claus comprades no hi ha res a vendre.

6.2 Mòdul de seguretat i usuaris

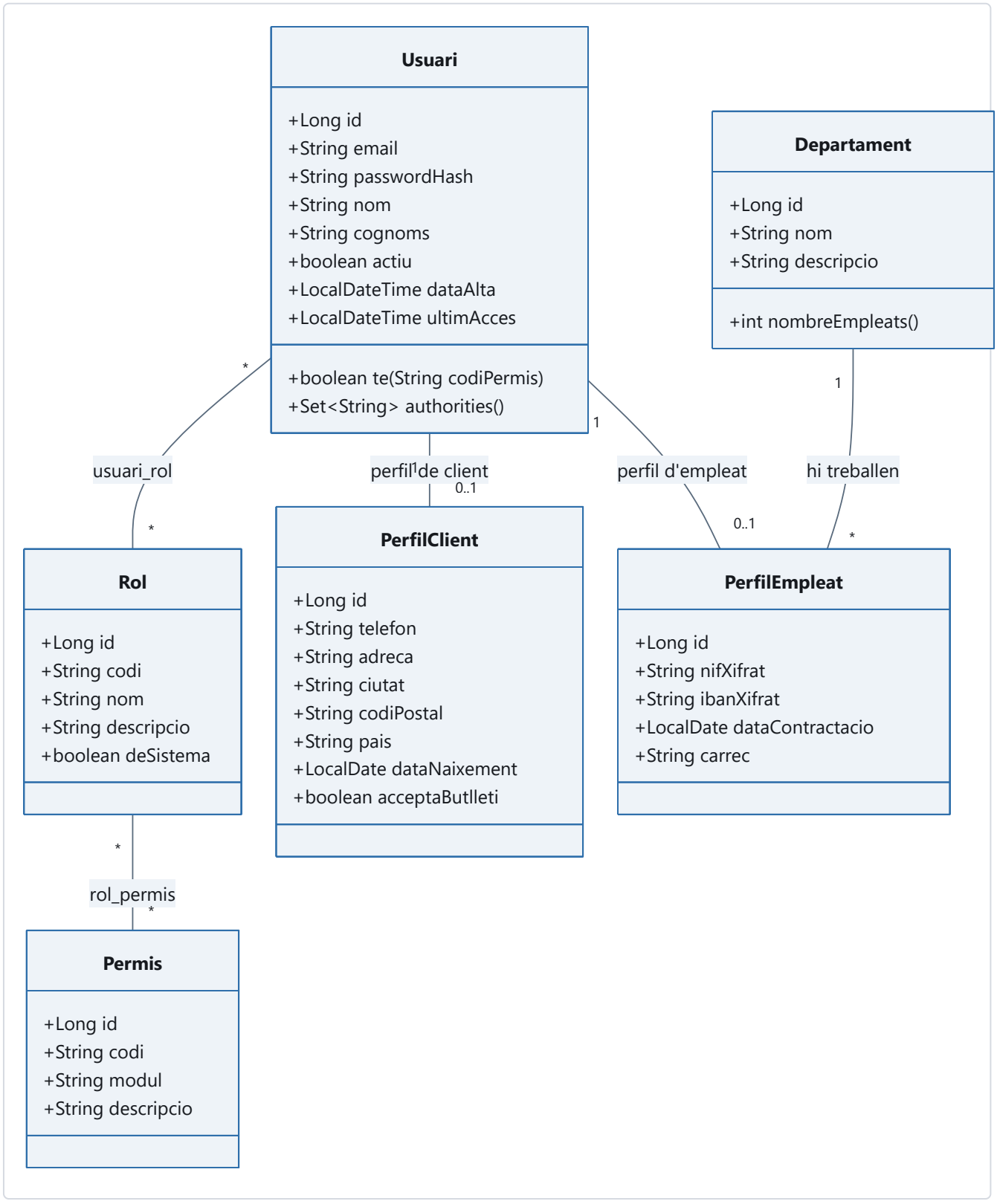


Figura 9. Seguretat i usuaris. Una sola taula d'autenticació amb dos perfils opcionals 1:1, i el control d'accés com a dades en tres taules.

6.3 Mòdul de catàleg

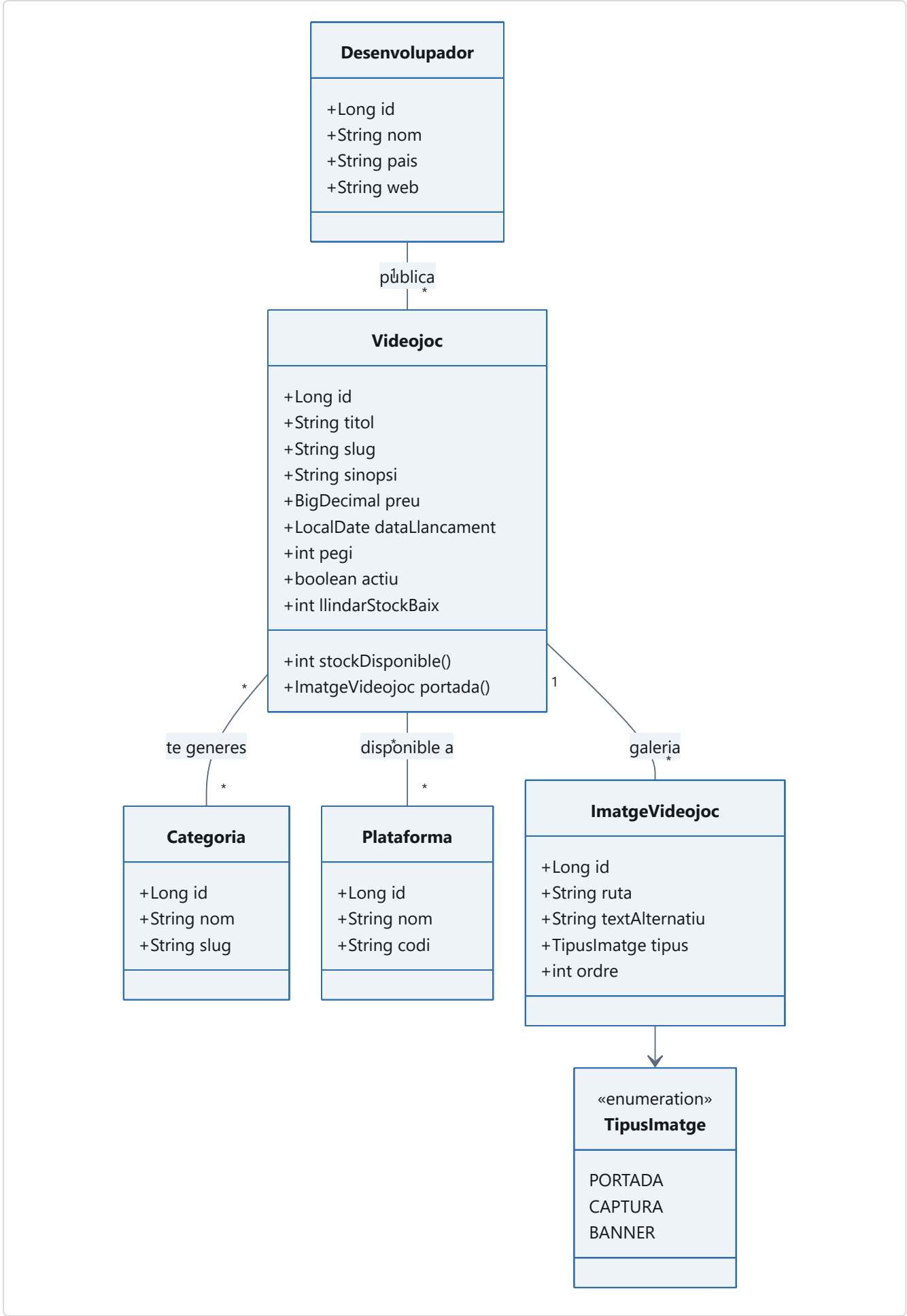

Desenvolupador
+Long id
+String nom
+String pais
+String web
publica
1
*
Videojoc
+Long id
+String titol
+String slug
+String sinopsi
+BigDecimal preu
+LocalDate dataLlancament
+int pegi
+boolean actiu
+int llindarStockBaix
+int stockDisponible()
+ImatgeVideojoc portada()
*
1
te generes
*
*
disponible a
*
*
galeria
*
Categoria
+Long id
+String nom
+String slug
Plataforma
+Long id
+String nom
+String codi
ImatgeVideojoc
+Long id
+String ruta
+String textAlternatiu
+TipusImatge tipus
+int ordre
«enumeration»
TipusImatge
PORTADA
CAPTURA
BANNER

Figura 10. Catàleg. Cada videojoc té una galeria d'imatges de la qual exactament una és de tipus PORTADA, garantit per una restricció única.

6.4 Mòdul d'aprovisionament: l'ordre de compra

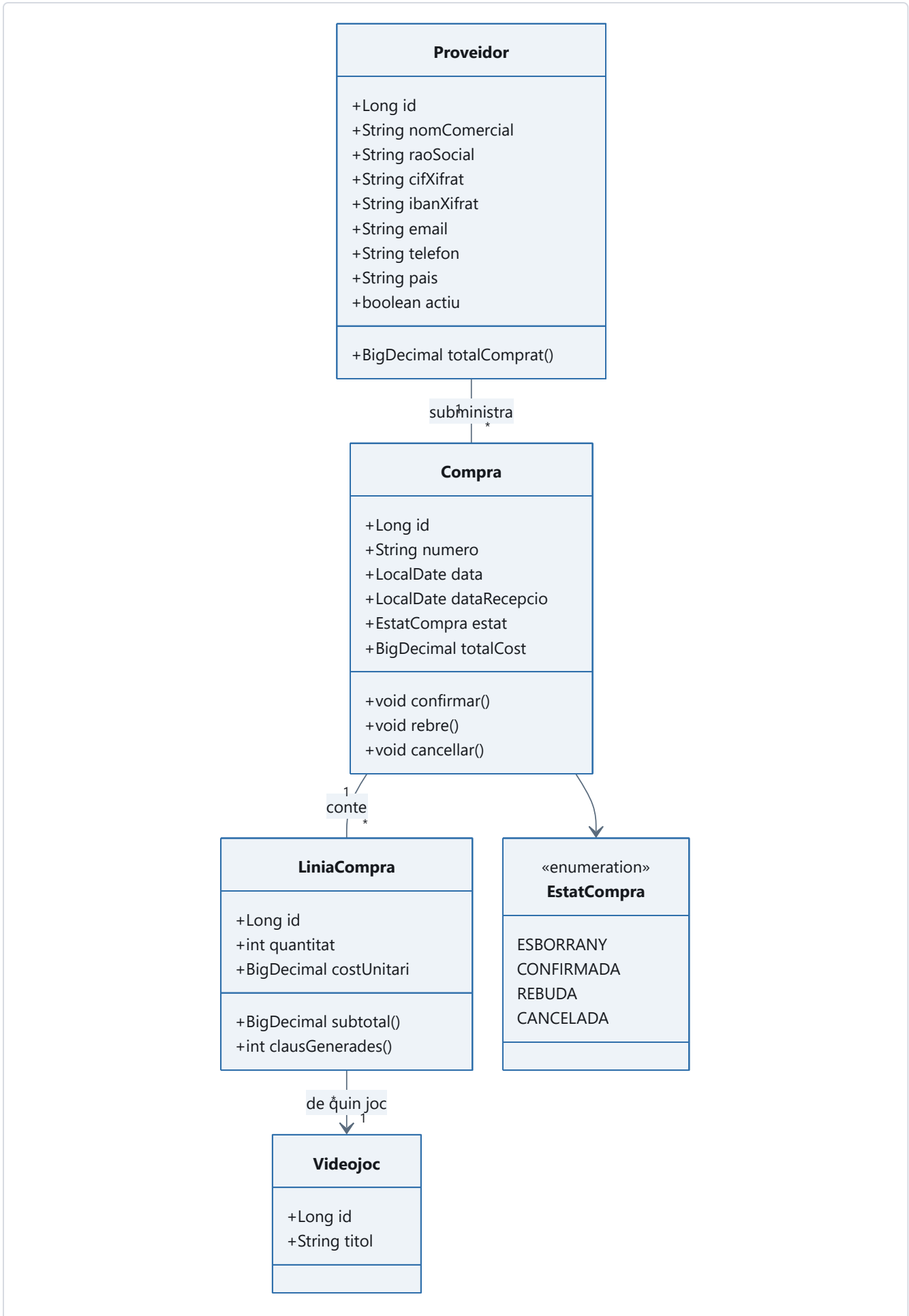


Figura 11. Capçalera i línies de l'ordre de compra. El mètode rebre() és el que dispara la creació de les claus de la figura següent.

6.5 Mòdul d'inventari: les claus de llicència

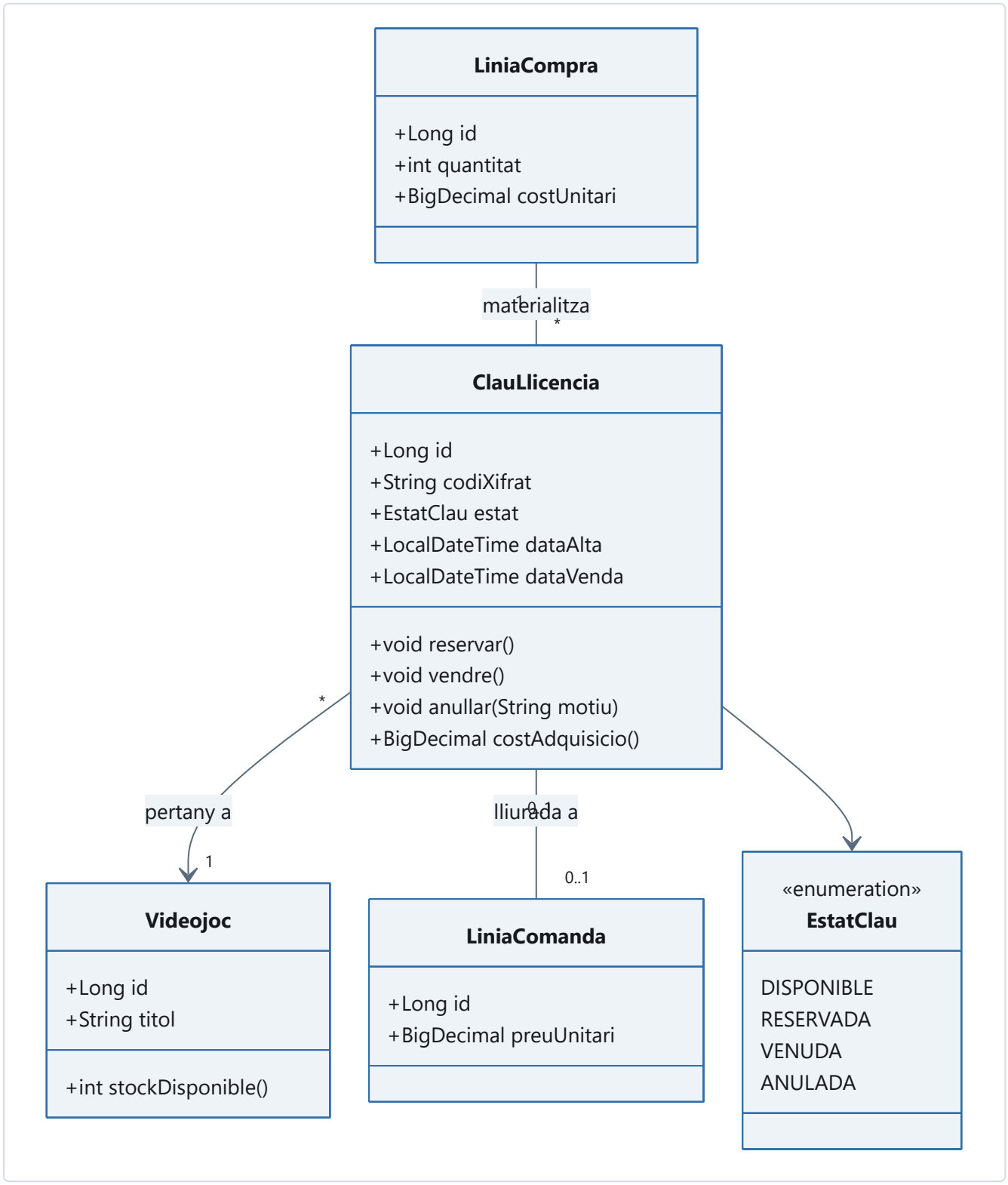


Figura 12. La clau de llicència és la unitat d'inventari i el punt on es tanquen les dues cares del negoci: apunta amunt a la línia de compra que la va crear i avall a la línia de comanda que la va vendre.

6.6 Mòdul de vendes i facturació

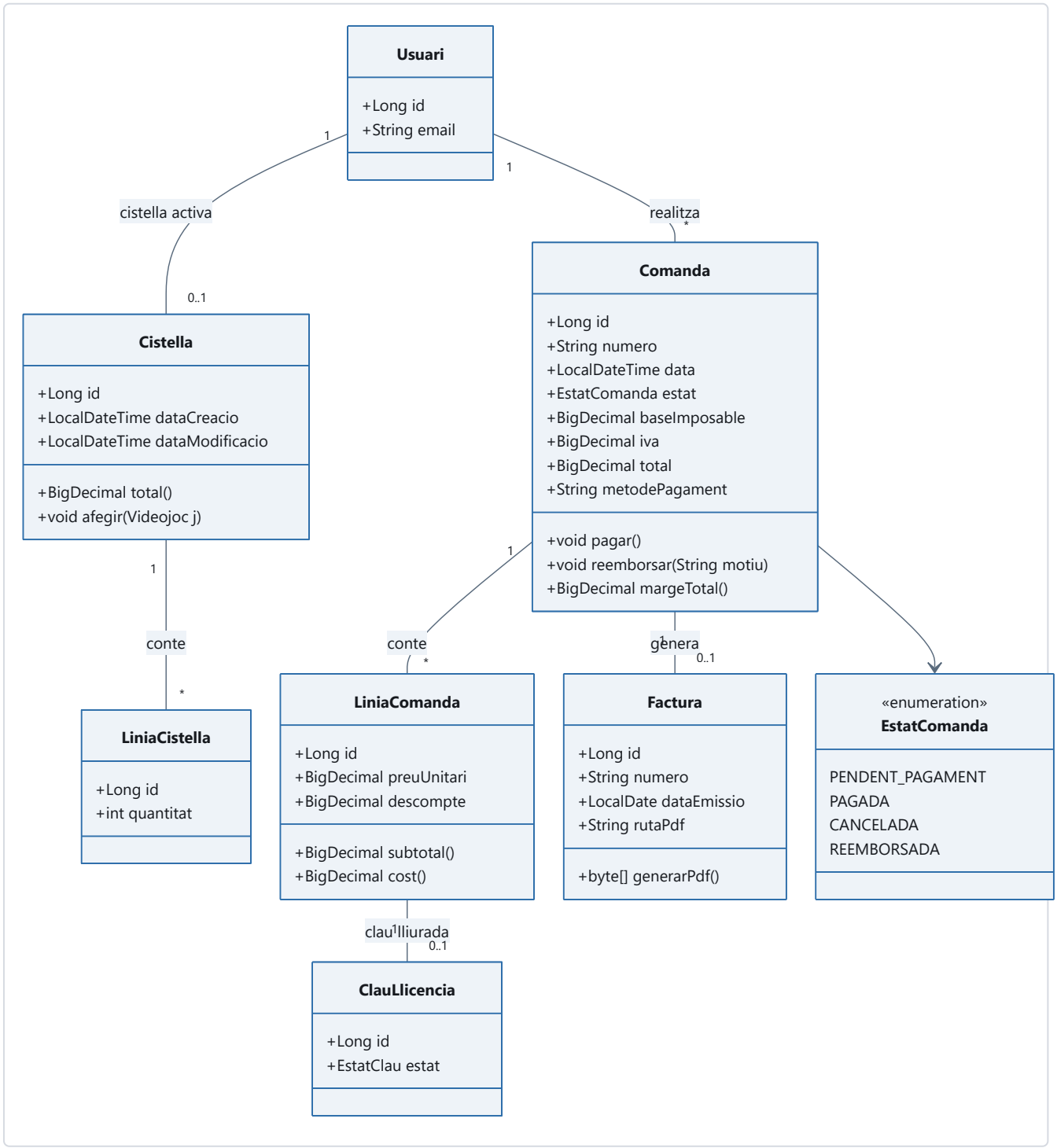


Figura 13. Vendes. La clau no s'assigna en afegir a la cistella sinó en confirmar el pagament; fins aleshores només es reserva, i això evita bloquejar estoc en cistelles que ningú no acabarà.

7. Diagrama d'arquitectura

L'aplicació és un monòlit modular de Spring Boot amb renderitzat al servidor. S'ha descartat separar un *frontend* en JavaScript i una API REST: multiplicaria la feina sense cap benefici per a un projecte d'aquesta mida, i l'enunciat ja fixa Thymeleaf com a motor de plantilles. El canal de WebSocket conviu amb l'HTTP tradicional dins del mateix procés, i les dues bases de dades es reparteixen la feina segons la naturalesa de la dada, no segons el mòdul.

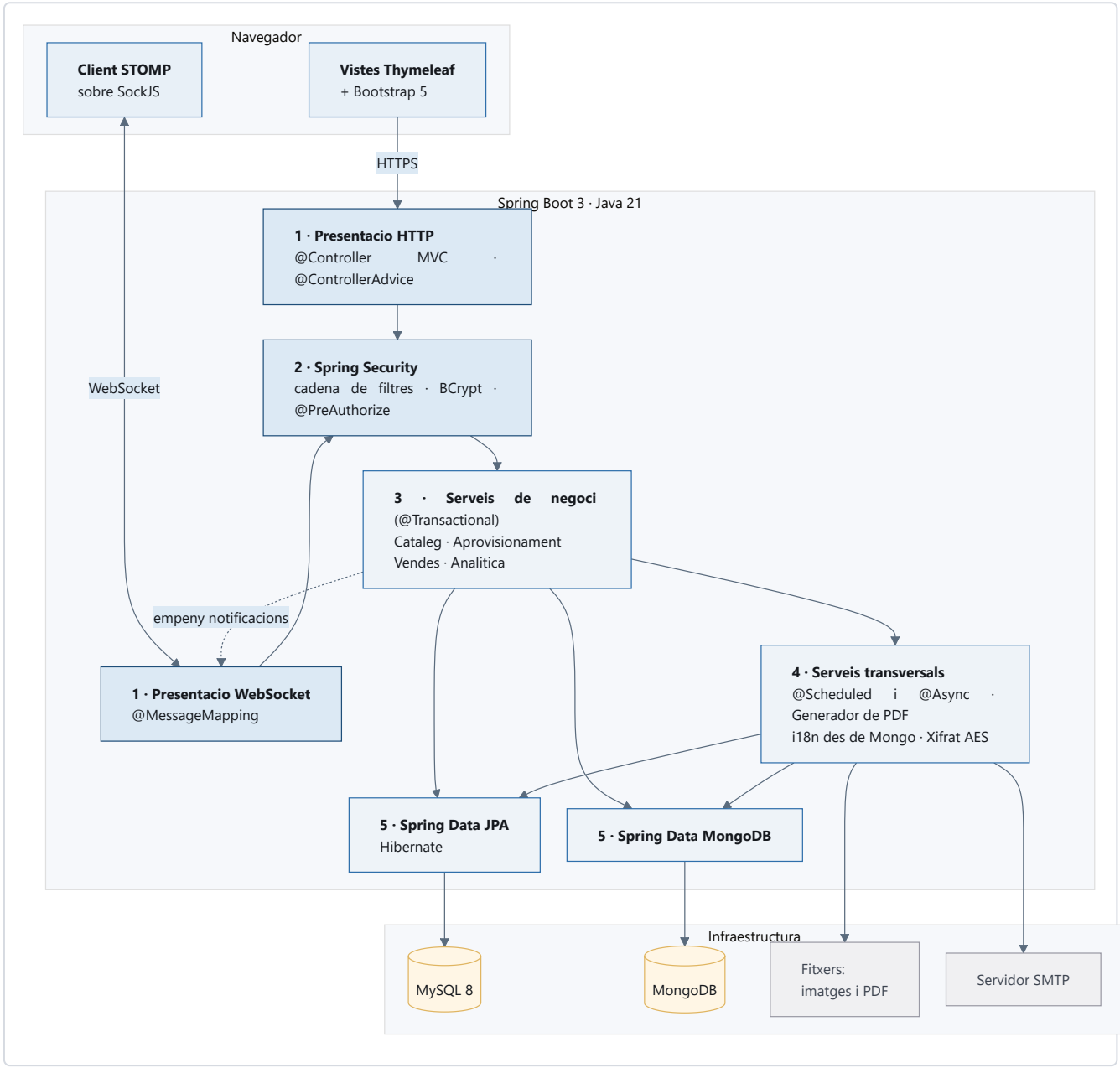


Figura 14. Arquitectura general en quatre capes. La fletxa discontínua és la clau del temps real: un servei de negoci (per exemple, una venda que esgota l'estoc) empeny un missatge cap al canal de WebSocket sense que el navegador hagi preguntat res.

8. Model de la base de dades MySQL

MySQL guarda tot el que és transaccional i relacional: el que ha de ser consistent, auditable i consultable amb unions. Totes les taules usen `BIGINT AUTO_INCREMENT` com a clau primària tècnica, joc de caràcters `utf8mb4` i el motor InnoDB per tenir claus foranes i transaccions reals. Els camps de tipus `varbinary` es desen xifrats amb AES-256-GCM. Per llegibilitat s'omet la longitud dels `varchar` i l'esquema es reparteix en sis vistes.

8.1 Seguretat i usuaris

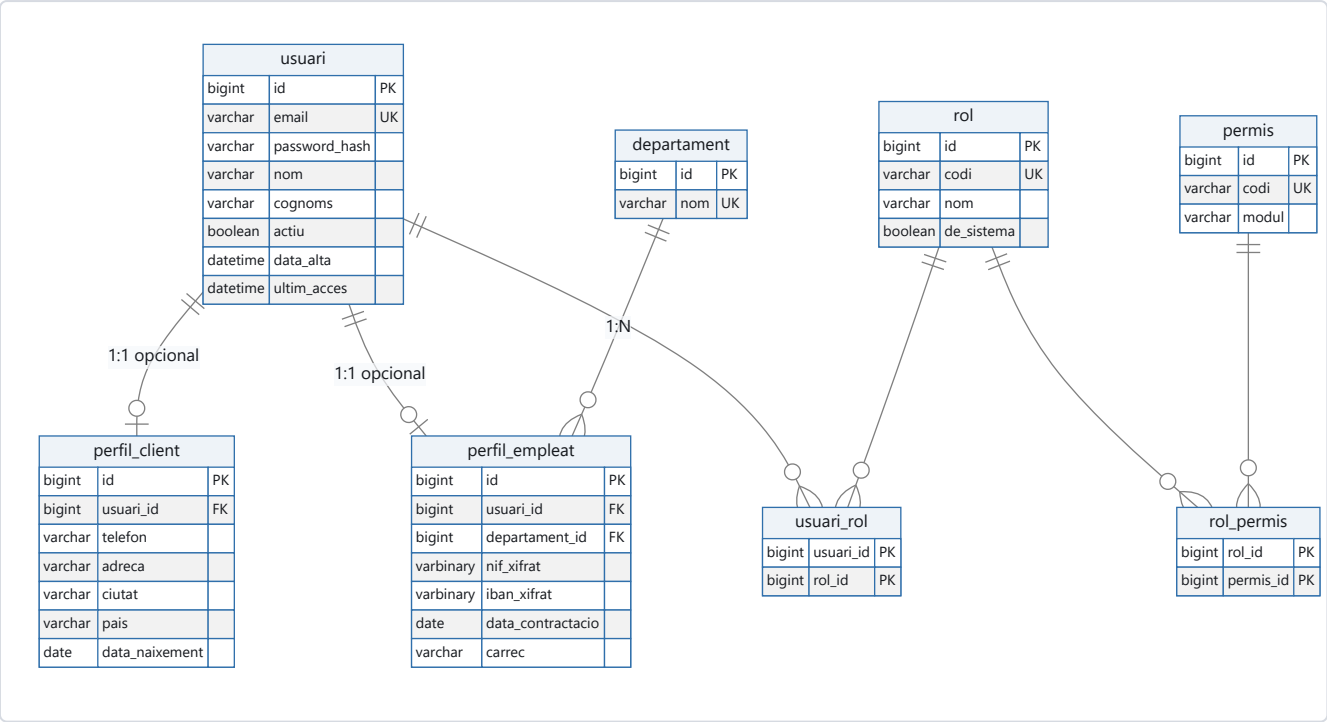


Figura 15. Esquema relacional de seguretat. Les dues taules pont, amb clau primària composta, implementen el control d'accés basat en permisos.

8.2 Catàleg: fitxa del joc i imatges

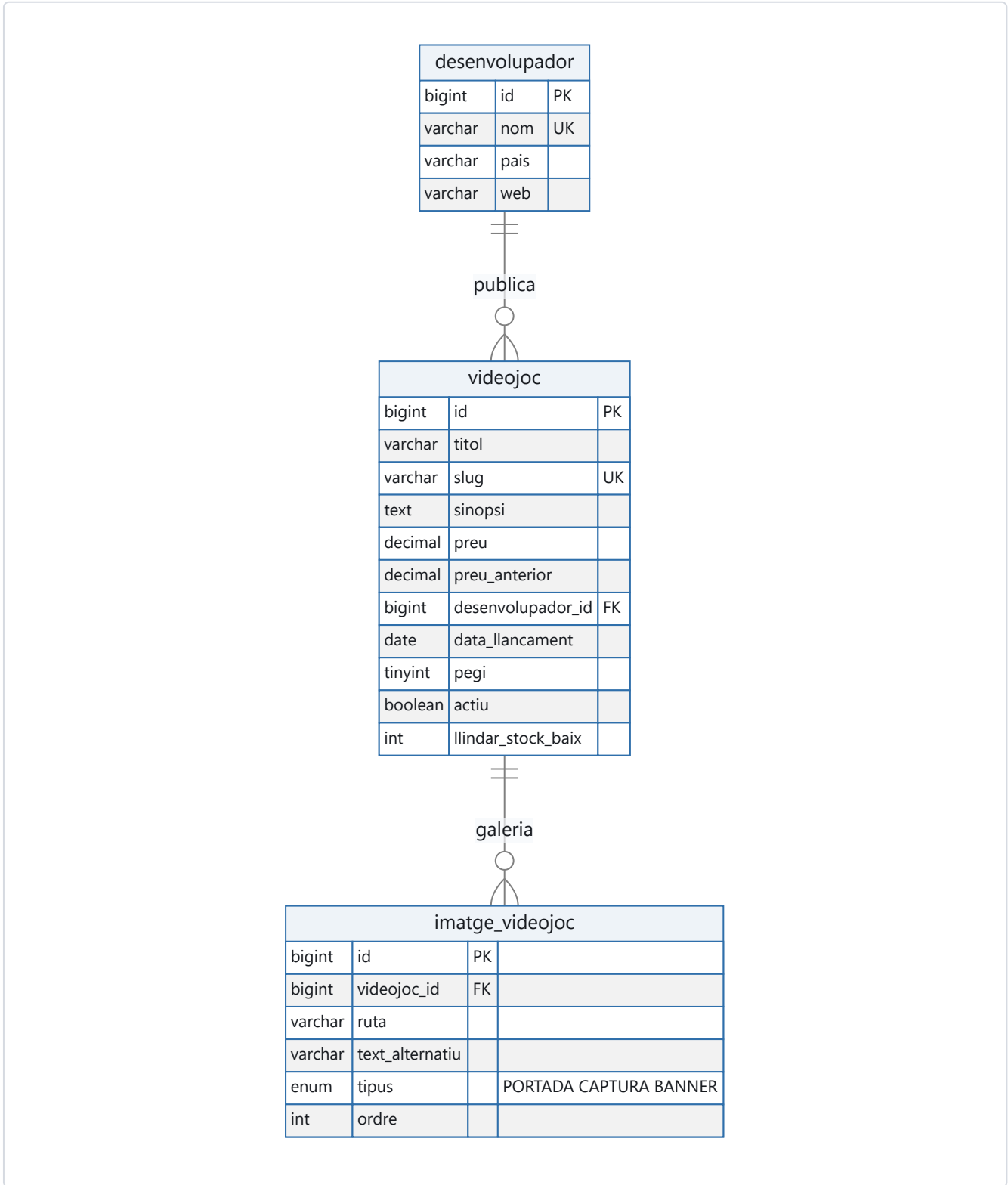


Figura 16. Nucli del catàleg. Restricció única: una sola fila amb tipus PORTADA per videojoc.

8.3 Catàleg: classificació

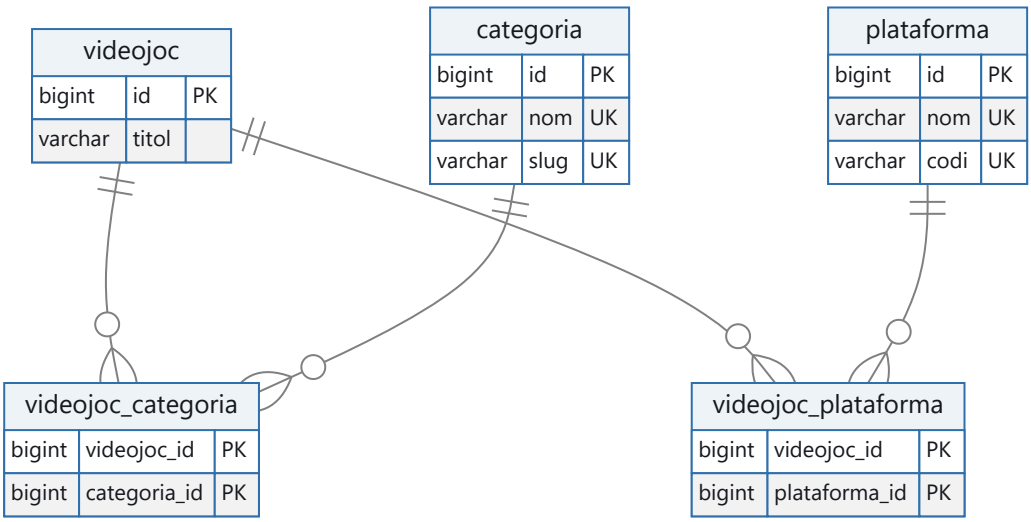


Figura 17. Les dues relacions N:M del catàleg, resoltes amb taula pont: un joc té diversos gèneres i està disponible en diverses plataformes.

8.4 Aprovisionament: ordres de compra

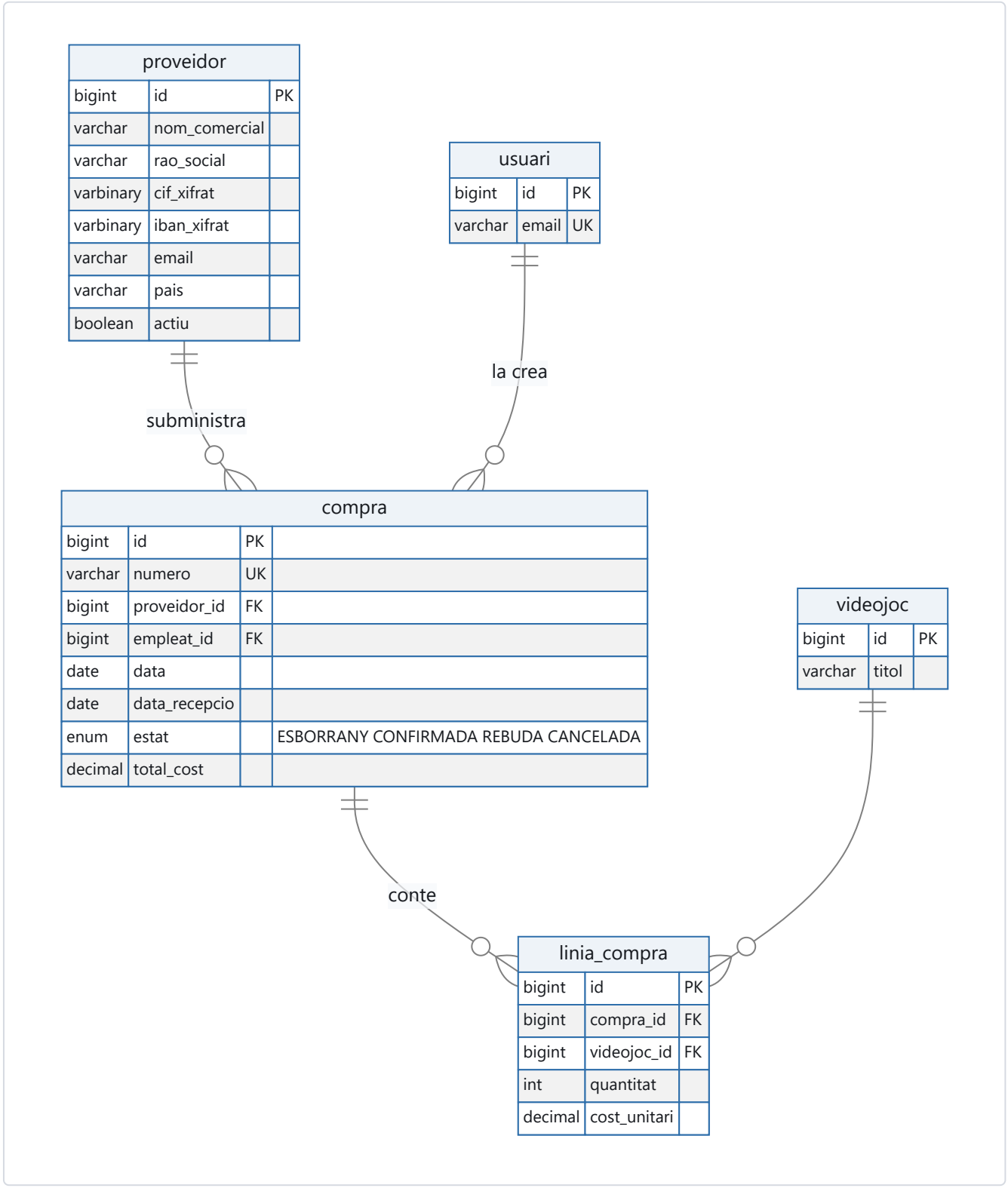


Figura 18. Ordres de compra al proveïdor. El camp empleat_id deixa constància de quin gestor d'inventari va crear cada ordre.

8.5 Inventari: claus de llicència

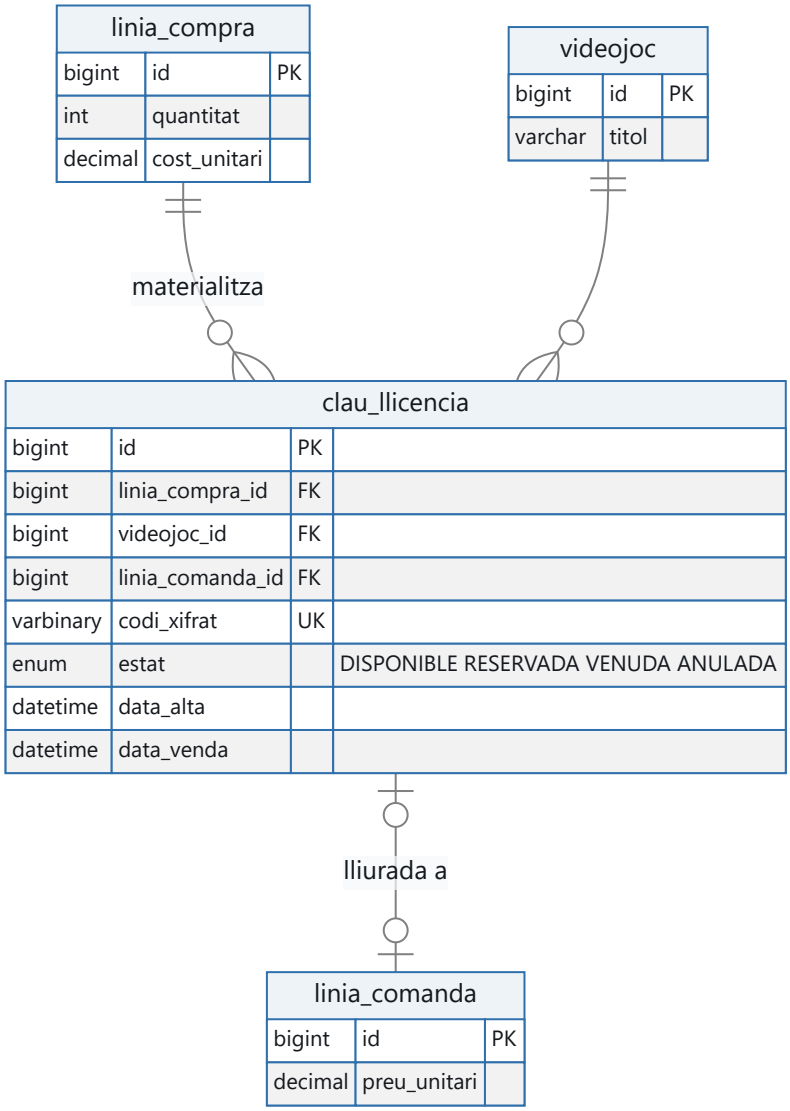


Figura 19. La taula d'inventari. El camp linia_comanda_id admet nuls: és nul mentre la clau no s'ha venut, i l'estoc d'un joc és el recompte de files amb estat DISPONIBLE.

8.6 Vendes i facturació

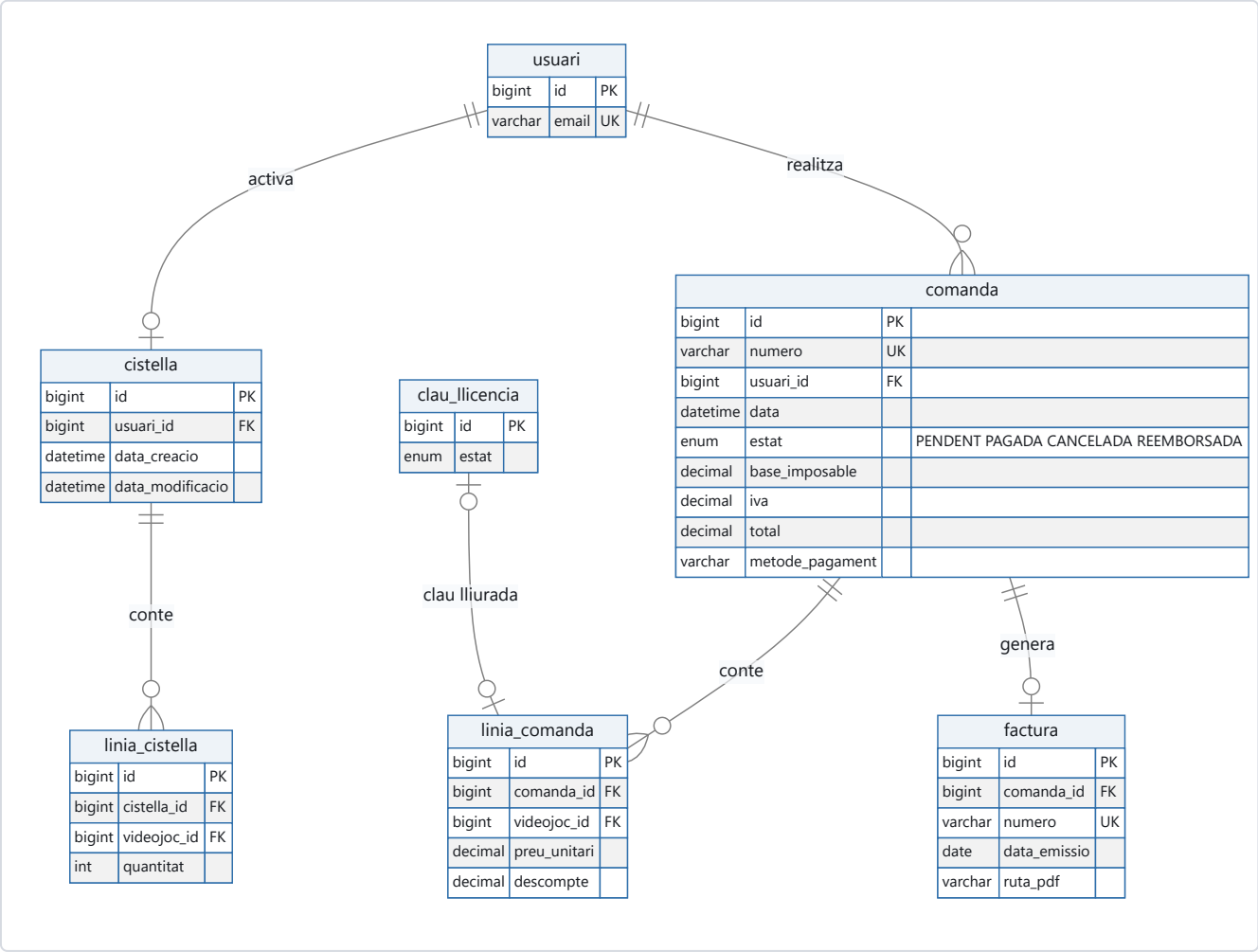


Figura 20. Vendes. La relació 1:1 entre comanda i factura té el camp comanda_id amb restricció única: una comanda no pot tenir dues factures. A `cistella`, `usuari_id` no admet nuls perquè no hi ha cistelles anònimes.

9. Model de dades a MongoDB

MongoDB no duplica res del que hi ha a MySQL. S'hi porta exactament el que és mal negoci per a una base de dades relacional: documents d'estructura variable, dades que es llegeixen moltíssim i s'escriuen poc, i registres que creixen sense límit i que no cal unir amb res. Són cinc col·leccions.

`traduccions`

Els textos de la interfície en els tres idiomes. Substitueix els fitxers `messages_*.properties` i és el que fa que l'idioma sigui dada i no codi. Índex únic sobre `(clau, idioma)`.

```
{
  "_id": ObjectId("..."),
  "clau": "cataleg.afegir_cistella",
  "idioma": "ca",
  "text": "Afegeix a la cistella",
  "modul": "cataleg",
  "modificat_el": ISODate("2026-10-22T10:14:00Z")
}
```

`preferencies_usuari`

La configuració de cada usuari, amb l'idioma com a camp principal. És el primer nivell de la resolució d'idioma descrita a l'apartat 3.4. Índex únic sobre `usuari_id`.

```
{
  "_id": ObjectId("..."),
  "usuari_id": 1042,
  "idioma": "ca",
  "moneda": "EUR",
  "tema": "clar",
  "notificacions": { "email": true, "websocket": true, "ofertes": false },
  "files_per_pagina": 20,
  "actualitzat_el": ISODate("2026-10-22T10:14:00Z")
}
```

`config_sistema`

Paràmetres globals editables per l'administrador sense tocar `application.properties` ni reiniciar: idioma per defecte, tipus d'IVA, llindar d'estoc baix, dades fiscals de la factura, correu de l'administrador.

```
{
  "_id": "idioma_per_defecte",
  "valor": "ca",
  "tipus": "ENUM",
  "valors_possibles": ["ca", "es", "en"],
  "descripcio": "Idioma que s'aplica quan l'usuari no te preferencia",
  "modificat_per": 1,
  "modificat_el": ISODate("2026-10-22T09:00:00Z")
}
```

`logs_activitat`

Auditoria de tota acció que modifica dades: qui, què, sobre quina entitat, quan i des de quina adreça. Va a MongoDB perquè creix indefinidament, mai no s'actualitza i no s'ha d'unir amb taules transaccionals. Es configura amb caducitat automàtica als dos anys i índexs sobre `usuari_id` i `moment`.

```
{
  "_id": ObjectId("..."),
  "moment": ISODate("2026-10-22T11:02:31Z"),
  "usuari_id": 7,
  "usuari_email": "gestor.productes@luduserp.cat",
  "accio": "PRODUCTE_EDITAR",
  "entitat": "videojoc",
  "entitat_id": 318,
  "canvis": { "preu": { "abans": 59.99, "despres": 44.99 } },
  "ip": "10.0.2.41",
  "resultat": "OK"
}
```

`estadistiques_cache`

Resultats precalculats cada nit per la tasca programada, de manera que el panell de l'administrador es dibuixi amb una sola lectura en comptes d'agregar tota la història de vendes a cada visita.

```
{
  "_id": "vendes_2026_10",
  "periode": { "tipus": "MES", "any": 2026, "mes": 10 },
  "ingressos": 48213.55,
  "cost": 31004.10,
  "marge": 17209.45,
  "unitats_venudes": 1187,
  "comandes": 904,
  "top_jocs": [
    { "videojoc_id": 318, "titol": "...", "unitats": 96, "ingres": 4319.04 }
  ],
  "per_dia": [ { "dia": 1, "ingressos": 1402.30, "unitats": 34 } ],
  "calculat_el": ISODate("2026-10-22T03:00:00Z")
}
```

Col·lecció	Qui hi escriu	Per què no és a MySQL
`traduccions`	Equip de desenvolupament	Document per idioma i clau, de forma variable i amb lectura massiva i cau; no té relacions.
`preferencies_usuari`	Qualsevol usuari	Estructura que creixerà amb opcions noves sense migracions d'esquema.
`config_sistema`	Administrador	Parelles clau-valor de tipus heterogeni; una taula relacional obligaria a columnes de tipus genèric.
`logs_activitat`	Sistema	Creixement il·limitat, només inserció, amb un camp de canvis de forma lliure.
`estadistiques_cache`	Planificador	Documents amb vectors imbricats que a MySQL caldria repartir en tres taules més.

10. Traçabilitat requeriment → disseny

Taula de verificació: per a cada punt de l'enunciat, on es resol dins d'aquest document.

Punt	Requeriment	Resolució al disseny
2.1	Diagrama de classes	Apartat 6, figures 8 a 13: vista de mòduls més cinc diagrames de classes amb atributs, operacions i multiplicitats.
2.1	Diagrama de casos d'ús amb perfils	Apartat 5, figures 1 a 7: un diagrama per actor, de l'accés sense sessió a l'administrador, més els processos automàtics.
2.1	Diagrama d'arquitectura	Apartat 7, figura 14.
2.1	Esquemes de MySQL i MongoDB	Apartat 8 (figures 15 a 20) i apartat 9.
3.1	Mínim tres perfils d'usuari	Cinc rols a l'apartat 2.1, amb matriu de permisos a 2.2.
3.2	Mínim quatre CRUD amb taules relacionades	Sis CRUD a l'apartat 3.2, cadascun amb les taules que relaciona i els filtres del llistat.
3.2	Validacions de servidor i de client	Final de l'apartat 3.2: Bean Validation i validadors de negoci al servidor; HTML5 i JavaScript al client.
3.3	Mínim un PDF professional	Tres PDF a l'apartat 3.3: factura, albarà de recepció i informe mensual, amb Thymeleaf i OpenHTMLtoPDF.
3.4	Català, castellà i anglès amb idioma a MongoDB	Apartat 3.4 i col·leccions `traduccions` i `preferencies_usuari` de l'apartat 9.
3.5	Sockets o WebSockets	Apartat 3.5: notificacions en temps real a la safata de l'empleat i estoc viu a la fitxa del producte, amb STOMP sobre Spring i SockJS de reserva.
3.6	Threads o processos asíncrons	Apartat 3.6: cinc treballs en segon pla entre `@Scheduled` i `@Async`; figura 7.
3.7	Spring Security, BCrypt i xifrat de dades sensibles	Apartat 3.7: BCrypt amb cost 12 i AES-256-GCM sobre NIF, IBAN, CIF i codi de clau de llicència.
4.1	Spring Boot, Thymeleaf, MySQL i MongoDB modulars	Apartat 4 i figura 14.
4.2	MVC i patrons de disseny	Apartat 4, fila de patrons: MVC, Repository, DTO, Strategy, Observer i Builder.
4.3	Rendiment, errors i registres	Apartat 4, files de rendiment i de registres; col·lecció `logs_activitat`.
4.4	Interfície clara i coherent	Apartat 4, fila d'interfície.
4.5	Documentació final	Apartat 4, última fila, i apartat 11.

11. Repartiment de tasques

Els mòduls s'han delimitat de manera que es puguin desenvolupar en paral·lel un cop estiguin fetes les peces compartides. La proposta és atacar-ho en tres onades:

1. **Base comuna** (tot el grup, primer): projecte, Docker Compose amb les dues bases de dades, entitats i migracions, Spring Security amb el model de rols i permisos, plantilla base de Thymeleaf i el `MessageSource` contra MongoDB. Res del que ve després no es pot provar sense això.
2. **Mòduls en paral·lel**: catàleg, aprovisionament i inventari, vendes i facturació, i analítica.

3. **Transversals i tancament**: traduccions completes als tres idiomes, PDF, tasques programades, proves, documentació i preparació de la defensa.

Membre	Mòduls assignats	Responsabilitats transversals
_____	Seguretat i usuaris · empleats i departaments	Spring Security, xifrat, registres d'auditoria
_____	Catàleg · imatges i classificació	Plantilla base, interfície i coherència visual
_____	Aprovisionament, inventari i proveïdors	Tasques programades i vigilància d'estoc
_____	Vendes, cistella i facturació	Generació de PDF
_____	Analítica i estadístiques	WebSockets i notificacions, multiidioma

Les caselles de membre es deixen en blanc perquè el grup les ompli. El repartiment no és estanc: cada integrant és el responsable del seu mòdul, però tots han de poder defensar el conjunt davant del tribunal.